Ders 8 Genişletilmiş Notu: Metin İşleme Algoritmaları ve Bilgiye Erişim Modelleri

Ders 8 Genişletilmiş Notu: Metin İşleme Algoritmaları ve Bilgiye Erişim Modelleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın. Altyazıda okunmayan bazı slayt bilgileri (LCS örneğindeki Y dizesi, küçük örnek belgeler, TF-IDF sonucu) kısa nottan alınmıştır; kısa not yazılırken bunlar kareden okunmuştur.

İçindekiler

- 1. Dersin Başında: Kelime Temsili Kaynakları ve Öğrenme Beklentisi
- 2. Metin İşleme Algoritmaları Hangi Sorulara Cevap Verir?
- 3. Ağaçların Bağlı Gösterimi
- 4. Trie Ağaçları
- 5. Sonek Ağacı (Suffix Tree)
- 6. En Uzun Ortak Alt Dizi (LCS)
- 7. En Uzun Ortak Alt Dize
- 8. Minimum Düzenleme Uzaklığı
- 9. Yazım Hatası Düzeltme Üzerine Soru-Cevap
- 10. Bilgiye Erişimin Kısa Tarihi
- 11. Bilgiye Erişim Sisteminin Mimarisi
- 12. Bilgiye Erişim Modellerinin Sınıflandırılması
- 13. Standart Boole Modeli
- 14. Genişletilmiş Boole Modeli
- 15. Vektör Uzayı Modeli
- 16. İndeksleme: Ters Çevrilmiş Dosya
- 17. Bulanık Mantık
- 18. Karma Min-Maks Modeli

1. Dersin Başında: Kelime Temsili Kaynakları ve Öğrenme Beklentisi

Bir öğrenci, önceki derste anlatılan kelime temsili (word representation) slaytlarının hangi kitaptan alındığını sordu. Kendisi kitaba bakmış, konuyu bulamamıştı.

Soru: Word representation slaytları hangi kitaptan?

Cevap: Hiçbir kitaptan değil. Hocanın elindeki ders kitaplarında bu konular yok, çünkü bunlar son yılların konuları. BERT gibi yöntemler kitapların eski baskılarına girmemiş. “Word representation” ve “word2vec” diye aranınca bir sürü kaynak çıkıyor: tutorial’lar, başka üniversitelerin ders slaytları, Medium yazıları. Anahtar kelimeyi bilmek yetiyor.

> **Hocanın vurgusu:** Bu derste görülen her şeyi yüzde yüz akılda tutarak çıkmak zordur. Kullanılmayan şey unutulur. Beklenti şudur: bir yerde duyduğunuzda ya da bir şeye ihtiyacınız olduğunda “böyle bir şey görmüştük, adı şuydu, bir inceleyeyim, benim problemime uyar mı” diyebilmek.

Hoca bugün iki slayt destesi olduğunu söyledi: önce metin işleme algoritmaları, sonra bilgiye erişim (information retrieval).

2. Metin İşleme Algoritmaları Hangi Sorulara Cevap Verir?

Hoca, konu başlığında “kelime” ya da “metin” demesinin önemli olmadığını hatırlattı. Kelime temsili slaytında olduğu gibi, bir işlem kelime, cümle ya da belge (document) düzeyinde yapılabilir. Bu bölümdeki algoritmalar üç soruya cevap verir:

- Elimdeki kelimeleri nasıl bir veri yapısında saklamalıyım ki onlara hızlı erişeyim? Çok tekrar eden kalıplar için boşuna bellek harcamayıp ortak kısımları ortak kullanabilir miyim?
- Elimde çok uzun dizeler (string) varsa bunların içindeki en uzun eşleşmeyi nasıl bulurum?
- Bir kelime yerine yanlışlıkla başka bir kelime yazılmışsa ikisi arasındaki uzaklığı nasıl ölçerim?

Son soru için hocanın örneği şu: “kabak” yazacakken yanlışlıkla K yerine T’ye basılıp “tabak” yazılmış olabilir. Burada ölçülen şey anlam benzerliği değil, sayısal bir mesafedir: kaç harf yanlış yazılmış, kaç harf eksik yazılmış. Mesafe küçükse yazılmak istenen kelime bulunabilir. Hoca bu yüzden “benzerlik demiyorum” diye özellikle uyardı.

3. Ağaçların Bağlı Gösterimi

Trie’ye geçmeden önce hoca ağaçların bellekte nasıl tutulabileceğini gösterdi. Örnek ağaçta kök B, ara düğüm D, yapraklar A, C, E ve F’dir. Kök B’nin üç çocuğu vardır: A, D, F. D’nin iki çocuğu vardır: C ve E.

Bağlı dizilerde olduğu gibi ağaçların düğümleri de birbirine bağlantılarla (link) bağlanabilir. Her düğüm bellekte üç gözle gösterilir:

- **Sol göz:** düğümde tutulan bilgi.
- **Orta göz:** bu düğüme nereden gelindiği, yani onu çağıran düğümün adresi. Kökte boştur.
- **Sağ göz:** bu düğümden gidilebilecek yerler.

Hoca gösterimi adım adım izledi. B’nin sol gözünde B yazar, orta gözü boştur. Sağ gözü, üç çocuğu tutan yapıyı gösterir. Birinci çocuk A’dır: sol gözünde A vardır, orta gözü B’yi gösterir, sağ gözü boştur çünkü A’dan gidilecek yer yoktur. D’nin orta gözü yine B’yi gösterir, sağ gözü ise iki çocuğu olan C ve E’yi gösterir. C ve E’nin orta gözleri D’yi gösterir. F’nin orta gözü B’yi gösterir.

Bu yapı sayesinde adresler izlenerek kimin kimin altında, kimin kimin üstünde olduğu kolayca çıkarılır.

4. Trie Ağaçları

4.1 Tanım ve kurallar

“Trie” adı “retrieval” kelimesinin ortasındaki harflerden gelir. Bir trie ağacında bir metin, bir cümle ya da bir sözlük saklanabilir. Örneğin TDK sözlüğünün kelimeleri, anlamları olmadan yalnız kelime olarak tutulabilir.

Kurallar şunlardır:

- Kök düğüm her zaman boş bir dizeyle başlar.
- Kökten herhangi bir yaprağa giden tek bir yol vardır.
- Her düğümden, aynı harfi taşıyan yalnız bir dal çıkabilir.
- Dallarda ilerledikçe geçilen harfler birleşir. Yaprağa gelindiğinde bir kelime elde edilir.

Avantajı, bir metni ararken metnin boyu kadar işlem gerektirmesidir. Kelime aranıyorsa gereken adım sayısı kelimenin uzunluğu kadardır, cümle aranıyorsa cümledeki kelime sayısı kadardır. Ayrıca belleği verimli kullanır.

4.2 Slayttaki örnek

Saklanan dizeler AEEF, AD, BBFE, BBFG ve C’dir. Kökten başlayıp herhangi bir yol izlenince mavi yapraklarda bu dizelerden biri elde edilir.

AEEF ile AD aynı “A” ile başlar. Bunlar için kökten iki ayrı “A” dalı açılabilirdi, ama o zaman aynı dal iki kez kullanılmış olurdu. Trie bunu engeller: daha önce açılmış bir dal varsa o yoldan devam edilir.

4.3 Dal sayısı ve derinlik

Kökten en fazla, alfabedeki harf sayısı kadar dal çıkar. Türkçe için hocanın cevabı şöyledir: alfabede 29 harf var, ama 29 dal çıkmaz. Yumuşak G ile başlayan Türkçe kelime olmadığı için en fazla 28 dal çıkar.

Ağacın derinliği sözlükteki en uzun kelimenin harf sayısı kadardır. Düğümlerde karakter yerine kelime tutulup cümleler ağaca yerleştiriliyorsa derinlik en uzun cümlenin kelime sayısı kadar olur.

4.4 Sıkıştırılmış trie

Slayttaki örnekte E’den E’ye, E’den F’ye giden yolda başka hiçbir dallanma yoktur. Bu yüzden bu zincir “EEF” etiketli tek bir kenar olarak yazılabilir. Aynı şekilde BB ile başlayan başka bir kelimede ayrım BBF’den sonra oluştuğu için “BBF” de tek kenar olur. Böylece ağaç daha az karmaşık hale gelir.

4.5 Dizi tabanlı gösterim

Trie yapıları veritabanlarında kayıtları yerleştirmek için de kullanılabilir. Hocanın gösterdiği yapıda alfabe A, E, I, P, R harflerinden oluşur. Her düğüm, alfabedeki her harf için bir göz içeren bir dizidir. Bir harf okununca o harfin gözünden bir sonraki diziye gidilir.

“RAP” kelimesi için önce kök dizide R gözü bulunur. Oradan gidilen dizide ikinci harfin gözü, sonrakinde P’nin gözü bulunur. RAP artık burada saklıdır. Gözlerde adresler tutulduğu için geriye doğru izlenerek köke dönülebilir. Bu gösterimde de kullanılmayan gözler çıkarılarak sıkıştırma yapılabilir.

4.6 İkili ağaç olarak gösterim

Trie, ikili ağaç (binary tree) yapısında da tutulabilir. İkili ağaçta bir düğümden en fazla iki dal çıkar:

- **Sağ bağlantı:** aynı seviyedeki bir sonraki harfi gösterir.
- **Sol bağlantı:** bir alt seviyeye, yani kelimenin bir sonraki harfine iner.

Arama şöyle işler: aynı seviyede aranan harf bulunana kadar sağ bağlantılarla ilerlenir. Harf bulununca sol bağlantıyla bir alt seviyeye inilir ve ikinci harf için aynı işlem yapılır. Hoca bunu “sanki özyinelemeli (recursive) bir fonksiyon çalıştırılmış gibi” diye anlattı.

4.7 Elle örnek: “ab”, “abla”, “aba”

Kökten sırayla A ve B inilir. Bu noktada “ab” kelimesi biter. “abla” gelince aynı A ve B dalları kullanılır, ama artık B’den devam eden bir kelime çıktığı için ağaç yenilenir: “ab”nin bittiğini göstermek için B’nin altına boş bir dal konur, yanına L dalı eklenir. L’nin altına A gelir ve “abla” oluşur. “aba” gelince aynı yoldan B’ye inilir ve üçüncü bir dal olarak A eklenir.

```
(kök)
 └─ a
     └─ b
         ├─ (boş)   -> "ab"
         ├─ l
         │   └─ a   -> "abla"
         └─ a       -> "aba"
```

5. Sonek Ağacı (Suffix Tree)

5.1 Ne işe yarar?

"Suffix" son ek demektir. Sonek ağacı trie'ye benzer, ama tam da aynı değildir. Kullanım alanları şunlardır:

- **Biyolojik veriler:** En çok burada kullanılır. Bir DNA dizisinde genin bir yerde bozulduğu düşünülüyorsa bozulmanın nerede olduğu aranır.
- **Arama motorları:** "okul" aranırken "okullar" gibi ek almış kelimelerin geçtiği belgelerin de gelmesi istenir. Sonek ağacı üzerinde ilerlenerek okuldan türetilmiş kelimeler bulunur.
- **Veritabanları ve derleyiciler (compiler).** İşletim sistemlerinde de kullanıldığı yazıyor, ama hoca hangi aşamada kullanıldığını bilmediğini söyledi.

Asıl amaç alt dize (substring) bulmaktır. "okullardan" içinde "okul" geçiyor mu? "ABCADAC" içinde "BCA" kalıbı var mı, "ADC" kalıbı var mı?

Karmaşıklık için hoca şunu söyledi: m uzunluğundaki bir dizeyi ağaçta aramanın süresi dizenin boyu kadardır. n uzunluğundaki bir Q alt dizesini aramak da alt dizenin boyu kadar sürer.

> **Ek bilgi (derste anlatılmadı):** Sonek ağacının alt dize aramaya yaramasının nedeni şudur: bir dizenin her alt dizesi, o dizenin bir sonekinin başlangıcıdır. Bütün sonekler ağaçta olduğu için kökten başlayıp alt dizenin harfleriyle inmek, alt dizenin geçip geçmediğini söylemeye yeter.

5.2 Örnek: ABAB

Dizenin sonuna, bittiğini gösteren `$` işareti konur. Dizenin sonekleri, en kısadan başlayarak `$`, `B$`, `AB$`, `BAB$` ve `ABAB$` şeklindedir.

Hoca ağacı en uzun sonekten başlayarak kurdu:

1. `ABAB$` için kökten tek bir kenar çıkar ve dize sonlanır.
2. `BAB$` A ile değil B ile başlar. Kökten yeni bir dal çıkar. Aralarda başka dallanma olmadığı için bu da sıkıştırılmış tek kenar olarak yazılır.
3. `AB$` için yeni bir A dalı açılmaz, çünkü AB zaten var. AB'den sonra bir düğüm eklenir, buradan `$` dalı ayrılır. Önceki kelime artık AB, sonra AB$ diye devam eder.
4. `B$` için de yeni bir B dalı açılmaz. B'den sonra düğüm eklenir ve `$` dalı çıkar.
5. `$` için kökte daha önce çıkmış bir `$` olmadığı için yeni dal eklenir.

Sonuçta beş yaprak vardır, her biri bir soneki verir:

```
(kök)
 ├─ AB ─┬─ AB$    -> ABAB$
 │      └─ $      -> AB$
 ├─ B ──┬─ AB$    -> BAB$
 │      └─ $      -> B$
 └─ $             -> $
```

5.3 İkinci örnek: CACCA

Hoca tahtada önce başka bir dizeyle başladı, karıştırdı, "çok uzun koydum, kontrolümü kaybettim" diyerek daha kısa bir dize aldı. İkinci denemede soneklerini sondan başlayarak ekledi: `$`, `A$`, `CA$`, `CCA$`, `ACCA$`, `CACCA$`.

Altyazı bu kısımda tam izlenemiyor, ama anlatılan adımlar şu ağacı verir (ara adım eklendi, soneklerden hesaplanarak kuruldu):

```
(kök)
 ├─ $                    -> $
 ├─ A ─┬─ $              -> A$
 │     └─ CCA$           -> ACCA$
 └─ C ─┬─ A ─┬─ $        -> CA$
       │     └─ CCA$     -> CACCA$
       └─ CA$            -> CCA$
```

Kural aynıdır: tekrar eden bir kısım varsa o dal yeniden kullanılır, ayrım noktasında yeni dal açılır. Ekleme sırası sondan da olsa baştan da olsa sonuçta aynı ağaca ulaşılır.

5.4 Trie ile farkı

Trie'de ayrı ayrı dizeler gelir ve birbirine bağlı olarak ağaca eklenir. Sonek ağacında ise tek bir dize vardır ve bu dizenin sonekleri alınarak eklenir: baştaki harf atılır, kalan sonek eklenir, sonra yine baştaki harf atılır.

6. En Uzun Ortak Alt Dizi (LCS)

6.1 Alt dizi nedir?

Hoca "ACTTGCG" dizesi üzerinden hangi dizilerin alt dizi (subsequence) olduğunu sordu:

- "ACT": evet.
- "ATTC": evet. A, sonra iki T, sonra C alınır. Karakterler peş peşe olmak zorunda değildir.
- "T": evet. Tek bir T de alt dizidir.
- "ACTTGC": evet.
- "TTA": hayır. İki T alındıktan sonra sırada bir A gerekir, ama A o T'lerden öncedir. Sıralama bozulur.

Kural şudur: alt dizi çıkarırken karakterler peş peşe gelmese bile sıra bozulmamalıdır.

6.2 Problem

İki dize verildiğinde, her ikisinin de alt dizisi olan en uzun diziyi bulmak istenir. Slaytta iki DNA dizisi için bu dizinin uzunluğu 13 çıkıyor. Hoca, birden çok alt dizi çıkabileceğini ama bundan daha uzun olanının olmadığını söyledi.

Bu neden gerekli olabilir? Hocanın örneği çok isimli insanlardır. "Ali Hasan Hüseyin Yüksek" adlı biri formlara "Ali Hasan Yüksek", "Hasan Hüseyin Yüksek" ya da "Ali Yüksek" yazmış olabilir. Bunların aynı kişi olduğu bulunmak istenebilir. Elbette ülkede çok sayıda Ali Yüksek vardır. Hoca bu nedenle kararın, kişiye ait belgelerdeki başka anahtar kelimelerin de uyuşmasıyla desteklendiğini ekledi.

6.3 Algoritma

X dizesinin boyu m, Y dizesinin boyu n olsun. c tablosu uzunlukları, b tablosu yönleri (okları) tutar. 0. kolon ve 0. satır sıfırla doldurulur. Sonra iki döngüyle tablo doldurulur:

$$c[i,j] = \begin{cases} 0 & i = 0 \text{ ya da } j = 0 \\ c[i-1, j-1] + 1 & x_i = y_j \\ \max(c[i-1, j],\ c[i, j-1]) & x_i \neq y_j \end{cases}$$

- Karakterler aynıysa çaprazdaki değerin bir fazlası yazılır ve çapraz ok konur.
- Karakterler farklıysa üstteki değer ile soldaki karşılaştırılır. Üstteki soldakinden büyük ya da eşitse üstteki yazılır ve yukarı ok konur. Hoca "burası önemli, büyük eşit" diyerek eşitlik durumunu vurguladı. Değilse soldaki yazılır ve sol ok konur.

Tablo dolunca sağ alt köşedeki değer LCS'nin uzunluğudur.

6.4 Örnek: X = ACTTGCG, Y = ATTCGG

Burada $m = 7$, $n = 6$. Hoca birkaç hücreyi elle doldurdu:

- A ile A aynı: çapraz 0'ın bir fazlası, 1, çapraz ok.
- A ile T farklı: solda 1, üstte 0. Büyük olan 1 yazılır, sol ok konur.
- C satırında C ile C aynı: çapraz 1'in bir fazlası, 2, çapraz ok.
- C ile G farklı: solda 2, üstte 1. Büyük olan 2 yazılır, sol ok konur.
- Son satırda G ile G aynı: çapraz 4'ün bir fazlası, 5.

Tablonun tamamı:

		A	T	T	C	G	G
	0	0	0	0	0	0	0
A	0	1↖	1←	1←	1←	1←	1←
C	0	1↑	1↑	1↑	2↖	2←	2←
T	0	1↑	2↖	2↖	2↑	2↑	2↑
T	0	1↑	2↖	3↖	3←	3←	3←
G	0	1↑	2↑	3↑	3↑	4↖	4↖
C	0	1↑	2↑	3↑	4↖	4↑	4↑
G	0	1↑	2↑	3↑	4↑	5↖	5↖

Sağ alt köşe 5'tir, yani en uzun ortak alt dizinin uzunluğu 5'tir.

6.5 Geri izleme

Uzunluk bulundu, peki çakışan dizi nedir? Sağ alt köşeden başlanır:

1. G ile G aynı: G yazılır, çapraza gidilir.
2. C ile G farklı: ok yukarıyı gösterir, yukarı çıkılır.
3. G ile G aynı: G yazılır, çapraza gidilir.
4. T ile C farklı: ok solu gösterir, sola gidilir.
5. T ile T aynı: T yazılır, çapraza gidilir.
6. T ile T aynı: T yazılır, çapraza gidilir.
7. C ile A farklı: ok yukarıyı gösterir, yukarı çıkılır.
8. A ile A aynı: A yazılır, çapraza gidilir ve sıfırıncı indise gelinir. İşlem biter.

Bulunan karakterler ters sırada okunur: LCS = "ATTGG", uzunluğu 5.

Hoca, tablonun sol üstten başlamak zorunda olmadığını, tersten de doldurulabileceğini ekledi. Bir de şunu söyledi: algoritma çok basit, ama uzun dizelerde gözle en uzun eşleşmeyi bulmaya çalışınca çok hata yapılır. Algoritma olmadan bunu bulmak gerçekten zordur.

7. En Uzun Ortak Alt Dize

İki dize trie mantığıyla aynı ağaca yerleştirilir. Slaytta bir dizenin dalları siyah, diğerininki kırmızıdır. Bir daldan hem siyah hem kırmızı geçiyorsa o dal iki dizede de vardır, yani ortak bir alt dizedir. Örnekte AB ve B böyle dallardır.

Slaytta aynı algoritmanın dinamik programlama kodu da vardı. Hoca ayrıntısına girmedi. Dinamikliği sağlayan kısmın, karakterlerin aynı ya da farklı olması durumunda ne yapılacağını gösteren satır olduğunu söyledi ve merak edenlerin bakabileceğini ekledi.

> **Ek bilgi (derste anlatılmadı):** Alt dize için karakterler peş peşe olmak zorundadır. Bu yüzden dinamik programlama kuralı LCS'den biraz farklıdır: karakterler aynıysa $c[i,j] = c[i-1,j-1]+1$, farklıysa $c[i,j] = 0$ olur. Sonuç, sağ alt köşedeki değer değil, tablodaki en büyük değerdir.

8. Minimum Düzenleme Uzaklığı

8.1 Tanım

Minimum düzenleme uzaklığı (minimum edit distance) iki dize arasındaki farkı sayısal olarak ölçer. Hocanın ilk örneği "aba" ile "abla"dır. Araya L girmişse bu bir eklemedir, L silinmişse bir silmedir. "abla" yerine "alya" yazılmışsa B silinip yerine L girmiş, L ile Y yer değiştirmiş olabilir.

Kullanım alanları: dize karşılaştırma, yazım denetimi (spell checking), biyoinformatik ve makine çevirisi (machine translation).

Üç işlem vardır:

- Silme (deletion).
- Ekleme (insertion).
- Yerine koyma (substitution).

8.2 Maliyetler ve “intention” ile “execution”

Bazı algoritmalarda her işlemin maliyeti 1’dir. Bu durumda “intention” dizesi “execution” dizesine şöyle çevrilebilir:

```
I N T E _ N T I O N
_ E X E C U T I O N
d s s   i s
```

Burada _ karşılığı olmayan yeri, d silmeyi, s yerine koymayı, i eklemeyi gösterir. İşlem sayısı 5’tir, yani uzaklık 5’tir.

Levenshtein metriğinde ise yerine koyma 2 maliyetlidir. Gerekçe şudur: T yerine X koymak, aslında T’yi silip X’i eklemektir, yani iki işlemdir. Bu durumda 3 yerine koyma 6 eder, 1 ekleme 7, 1 silme 8 eder. Uzaklık 8 olur.

> **Hocanın vurgusu:** Kaç silme, kaç ekleme yapıldığı kişiden kişiye değişebilir. Biri yerine koyma kullanır, biri önce ekleyip sonra siler. Yapılan işlemler farklı olabilir, ama maliyet, yani uzaklık hiçbir zaman değişmez.

8.3 Algoritma

D matrisinin 0. kolonu ve 0. satırı indisle doldurulur: $D(i,0) = i$, $D(0,j) = j$. Sonra özyineleme bağıntısıyla içi doldurulur:

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2 & x_i \neq y_j \\ 0 & x_i = y_j \end{cases} \end{cases}$$

Bir göz hesaplanırken üç yere bakılır. Yandaki iki gözün değerine 1 eklenir. Çaprazdaki gözün değerine, karakterler aynıysa 0, farklıysa 2 eklenir. Bunların en küçüğü yazılır. Değer nereden geldiyse oraya ok konur. Birden çok yönden aynı minimum geliyorsa birden çok ok konur. Çaprazdan gelen işlem yerine koymadır.

8.4 Örnek: matrisin doldurulması

Kaynak (source) “intention”, hedef (target) “execution”dır. Hoca matrisi alttan yukarı doğru çizdi. Bu da LCS’te söylediği gibi başlangıç yönünün serbest olduğunu gösteriyor.

Elle doldurulan gözler:

- **İlk göz (I, E):** Karakterler farklı, çapraz $0 + 2 = 2$. Yanlardan $1 + 1 = 2$ ve $1 + 1 = 2$. Hepsi 2 olduğu için buraya 2 yazılır ve üç yönlü ok konur. Çünkü bu 2 yerine koymadan da, silmeden de, eklemeden de gelmiş olabilir.
- **(I, X) gözü:** Yanlardan $2 + 1 = 3$ ve $2 + 1 = 3$, çapraz $1 + 2 = 3$. Değer 3, yine üç yönlü ok.
- **(E, C) gözü:** Aşağıdan $7 + 1 = 8$, soldan $5 + 1 = 6$, çapraz $6 + 2 = 8$. Minimum 6’dır, yalnız soldan gelmiştir. Tek ok konur.
- **(E, N) gözü:** Aşağıdan $8 + 1 = 9$, soldan $10 + 1 = 11$, çapraz $9 + 2 = 11$. Minimum 9’dur, yalnız aşağıdan gelmiştir.

Matrisin tamamı (ara adım eklendi, derste slaytta gösterilen matris formülle yeniden hesaplandı; elle doldurulan gözler hocanın değerleriyle aynı):

```
N   9   8   9  10  11  12  11  10   9   8
O   8   7   8   9  10  11  10   9   8   9
I   7   6   7   8   9  10   9   8   9  10
T   6   5   6   7   8   9   8   9  10  11
N   5   4   5   6   7   8   9  10  11  10
E   4   3   4   5   6   7   8   9  10   9
T   3   4   5   6   7   8   7   8   9   8
N   2   3   4   5   6   7   8   7   8   7
```

```
I   1   2   3   4   5   6   7   6   7   8
#   0   1   2   3   4   5   6   7   8   9
    #   E   X   E   C   U   T   I   O   N
```

Sağ üst köşedeki değer 8'dir. "intention" ile "execution" arasındaki uzaklık 8'dir.

8.5 Geri izleme

"8 de hangi işlemlerle 8?" sorusunun cevabı için sağ üst köşeden geri gidilir:

1. N ile N, O ile O, I ile I, T ile T aynıdır. Hiçbir işlem yapılmaz, çapraza gidilir.
2. N ile U farklıdır. Gözde üç ok vardır, istenen seçilebilir. Hoca yerine koymayı seçti: N yerine U (+2).
3. E ile C farklıdır. Burada tek ok vardır, soldan gelir: C eklenir (+1).
4. E ile E aynıdır, işlem yok.
5. T ile X farklıdır. Yine yerine koyma seçildi: T yerine X (+2).
6. N ile E farklıdır. Yerine koyma seçildi: N yerine E (+2).
7. Geriye I kaldı, karşılığı yoktur: I silinir (+1).

Toplam $2 + 1 + 2 + 2 + 1 = 8$.

Üç oklu bir gözde silme ya da ekleme seçilseydi başka bir yoldan ilerlenecekti, ama sonuç yine 8 olacaktı. Hoca, yerine koymanın iki işlemi bir arada yaptığı için daha az adımla ilerlediğini, silme ya da ekleme seçilirse yolda iki kutu daha geçileceğini söyledi.

Karmaşıklık hem zaman hem bellek için aynıdır. Hoca bunu "n kare karmaşıklığında bir algoritma" diye özetledi. Dizelerin boyları m ve n ise $O(mn)$'dir.

9. Yazım Hatası Düzeltme Üzerine Soru-Cevap

Aradan sonra iki öğrenci sorusu üzerine konu yazım düzeltmeye geldi.

Soru: Minimum edit distance'tan bahsettik. Kelimelerin düzeltilmesi konusuna gelecek miyiz?

Cevap: Hayır. Bunun için kendi algoritmanızı yazmanız gerekir. Hoca kendi öğrencisiyle yaptığı bir çalışmayı anlattı:

1. Türkçe karakterleri yanlış yazılmış bir kelime alındı. Örnek: "çocuk" kelimesinin c ile yazılmış hali.
2. Kelimedeki C'lerin yerine bütün C/Ç kombinasyonları üretildi.
3. Her aday, cümlede sağındaki ve solundaki kelimelerle birlikte arama motoruna gönderildi. Örnek cümle: "çocuk eve geldi".
4. Arama motorunun döndürdüğü sonuç sayıları karşılaştırıldı. Bir aday için yaklaşık 12.752, öteki için yaklaşık 3000 sonuç gelmişse az olanın yanlış yazılmış olduğu anlaşılır. O biçimi yazan başka insanlar da vardır, ama çoğunluk doğrusunu kullanır.
5. En çok sonuç dönen aday doğru kabul edildi. %90'a yakın doğru tespit yapıldı.

Başka çözüm yolları da var:

- Sonlu durum makineleri (finite state machines) kurmak.
- n-gram modelleriyle hangi karakterden sonra hangi karakterin gelme olasılığının yüksek olduğunu çıkarmak, bunları durumlar arası geçiş olasılığı olarak kullanmak.
- Saklı Markov modelleri (Hidden Markov Models, HMM).

Türkçe için geliştirilmiş hazır bir algoritma yoktur. Problem Türkçeye özgü olduğu için kendi verinizle çözmeniz gerekir. %100 başarı alınamasa da %90'ın üstüne çıkılabilir.

Başarısız olunan yerlerin nedeni bağlamdır (context). "Kıs" (sesi kısmak) ile "kış", "kus" ile "küs" karışabilir, çünkü ikisi de geçerli kelimedir. Yalnız karakter olasılıkları kullanılırsa bu durumlarda sorun çıkar. Bağlam bilgisi katılırsa problem çözülebilir. Kelime gömmeleri (word embeddings) yardımcı olabilir, çünkü gömme değerleri bağlamdaki kelimelerle birlikte oluşur. Ama hoca maliyete dikkat çekti: bir harf değiştirmek için koca vektörlerle işlem yapılmış olur.

Soru (proje üzerinden): Adres verisinde yanlış yazılmış "mahalle" ve "sokak" ifadelerini bulmaya çalışıyorum. Düzenli ifadeyle (regex) "SK" arayınca yanlış yazılmış "WSK" de geliyor, "eski" gibi kelimeler de. Ne yapabilirim?

Cevap: LCS denenebilir. Elinde “SK”, “sokak”, “SOK.” gibi doğru yazılabilme biçimleri var. Yanlış yazılmış WSK içinde bunlardan biri alt dizi olarak geçiyorsa fazladan olan harf hatalıdır ve çıkarılabilir. Hoca bunun aklına o an geldiğini, denemeden ne kadar başarılı olacağını söyleyemeyeceğini ekledi.

Öğrenci, PTT’nin il, ilçe, semt, mahalle ve posta kodu bilgisini Excel olarak paylaştığını, bulduğu mahalle adlarını bununla doğrulamak istediğini söyledi. Hoca buna bir öneri ekledi: sokak adlarına da erişilebilirse bulunan sokak adı, o mahalledeki sokaklardan en çok örtüşenle değiştirilebilir. Örneğin PTT’nin verdiği ad “Barbaros” ise ve kişi bir harfini yanlış yazmışsa, edit distance ile en yakın ad bulunur. Öğrenci, Excel’de en alt seviyenin mahalle olduğunu söyledi. Hoca bunun bir daha kontrol edilmesini önerdi.

10. Bilgiye Erişimin Kısa Tarihi

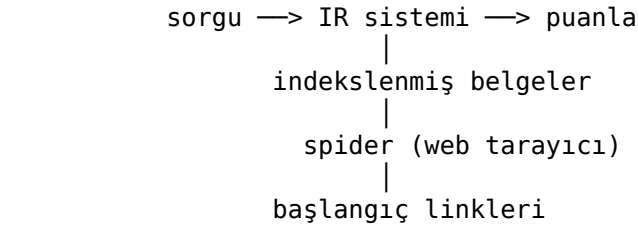
Bilgiye erişim sistemlerinin (information retrieval, IR) ilk örneği kütüphane veritabanlarıdır. İnternet olmadığı dönemde bir bilgiye ulaşmak için kütüphaneye gidilir ve bir anahtar kelimeyle başlanırdı. Örneğin fizikte akım teorisi aranıyorsa üzerinde alfabetik işaretler olan çekmecelerden A-K çekmecesi bulunur, kartotekslere bakılır, numara alınıp kütüphaneciye verilirdi. Kütüphaneci arşivden kitabı getirirdi. Aranan şey o kitapta yoksa işlem baştan alınırdı. Bu sistem tamamen insan gücüyle çalışıyordu.

Sonra arama motorları geldi:

- Metin tabanlı arama: Google, Yahoo, AltaVista. Önce tek anahtar kelime kullanıldı. Sonra birden çok anahtar kelime AND, OR gibi mantık bağlaçlarıyla bağlandı. Daha sonra cümle yazınca ilgili sayfalar gelmeye başladı.
- Çoklu ortam (multimedia) üzerinde arama: görsel ve video üzerinde arama. IBM’in QBIC (Query by Image Content) ürünü ve web için içerik tabanlı görsel-video kataloğu WebSEEk örnektir.
- Soru cevaplama (question answering) sistemleri: doğal dilde sorulan soruya cevap veren AnswerBus ve Ask Jeeves. Hakia adlı Türkçe sayfalarda çalışan bir arama motoru da bir dönem popüler olmuştu. Hoca hâlâ çalışıp çalışmadığını bilmediğini söyledi.

11. Bilgiye Erişim Sisteminin Mimarisi

Bir IR sistemi kapalı bir kutu gibi düşünülebilir. Arkasında indekslenmiş bir veritabanı vardır. Sorgu kelimesi ya da cümlesi sisteme gönderilir, sistem veritabanında arar ve sonuçları puanlanmış olarak döndürür. En ilgili belgeler ilk sırada gelir.



Hoca farkı kendi deneyiminden anlattı: ilk yıllarda aranan şey ikinci, üçüncü sayfada bulunabiliyordu, o yüzden üçüncü sayfaya kadar gidilirdi. Bugün çoğu zaman ilk üç sonuçta bulunuyor. Bunun sebebi iyi bir indeksleme mekanizması ve PageRank gibi iyi bir puanlama algoritmasıdır.

11.1 İndeksleme ve spider

İndekslenen şey yalnız kitaplar değil, bütün webdir: belge, HTML sayfası ya da başka bir şey olabilir. Bunun için bir spider yazılır. Spider’a başlangıç linkleri ve kaç seviye aşağı inebileceği (3, 5, 7 gibi) verilir. O sayfalardaki belgeler ve belgelerin içindeki kelimeler, konumları ve adresleriyle birlikte indekslenir.

Sayfaların çoğu dinamiktir, bu yüzden bir kez indekslemek yetmez. Her sayfayı aynı sıklıkta kontrol etmek de maliyetlidir. Gazete ve haber sayfaları gibi sık değişenler daha sık, nadiren değişenler daha seyrek indekslenir.

11.2 Sayfanın önemi

Bir sayfanın önemini ne belirler? Hocanın saydığı adaylar şunlardır:

- Sayfaya kaç sayfadan link geldiği.
- Sayfadan başka sayfalara link gidip gitmediği.
- Kullanıcıların sayfada ne kadar süre kaldığı.
- Sayfada sık sorgulanan kelimelerin geçmesi.
- Kaç kullanıcının sayfaya sürekli girdiği.

Bu algoritmaların tam olarak nasıl çalıştığı bilinmez, çünkü gizli tutulurlar. Kabaca neye bakılabileceği söylenebilir. En önemli ölçütlerden biri, sayfaya çok link verilmesidir. Hocanın örneği: hoca kendi sayfasından üniversitenin sayfasına link veriyorsa üniversitenin sayfası daha önemli olmalıdır.

12. Bilgiye Erişim Modellerinin Sınıflandırılması

Modeller matematiksel temellerine göre şöyle ayrılır:

- **Küme kuramsal (set-theoretic) modeller:** standart Boole, genişletilmiş Boole, bulanık bilgi erişimi (fuzzy retrieval).
- **Cebirsel (algebraic) modeller:** vektör uzayı modeli, LSA.
- **Olasılıksal (probabilistic) modeller.**

Bunların birbirine karşı avantajları ve dezavantajları vardır. Bazı modellerde terimler, yani kelimeler kullanılır, bazılarında kelimeler dışında başka özellikler kullanılır. Hoca en ilkel olandan başladı.

13. Standart Boole Modeli

Adından anlaşıldığı gibi bir terim belgede ya vardır ya yoktur. T terim kümesi, D belge kümesi, Q sorgudur. Sorgu terimleri AND, OR ya da ikisinin karışımıyla bağlanabilir. Model küme teorisine dayandığı için AND kesişim (intersection), OR birleşim (union) işlemidir.

Örnekte durak kelimeler (stop words) atılmış, dört terim çıkarılmıştır:

Terim	D1	D2	D3
Bayes' Principle	var		
Probability	var	var	var
Decision-making		var	
Bayesian Epistemology			var

Sorgu "probability AND decision-making" olsun. İki kelime yan yana yazılınca sistem zaten AND kabul eder.

- "probability" geçen belgeler: $S_1 = \{D_1, D_2, D_3\}$.
- "decision-making" geçen belgeler: $S_2 = \{D_2\}$.
- $S_1 \cap S_2 = \{D_2\}$. Sonuç D2'dir.

Avantajı: çok kolaydır, hiç formül kullanmaz.

Dezavantajı: sonuçlar sıralanmadan gelir. "probability" tek başına aranırsa üç belge de gelir, ama hangisinin öncelikli olduğu bilgisi yoktur. Terim ağırlıkları kullanılmaz. Bir belgede kelime 10 kez, ötekinde 1 kez geçse de ikisi aynıdır, oysa 10 kez geçenin önde gelmesi istenir.

> **Hocanın vurgusu:** Bu model bir "information retrieval" değil de "data retrieval" yapıyormuş gibi çalışır. Sonraki modeller bu dezavantajı ortadan kaldırmak için çıkmıştır.

14. Genişletilmiş Boole Modeli

14.1 Terim ağırlıklandırma

Genişletilmiş Boole modeli (extended Boolean model) kelimeleri ağırlıklandırarak kullanır, vektör uzayı modelinden yararlanır, sorgu ile belgeler arasındaki benzerliği hesaplayıp sonuçları sıralar.

Ağırlıklandırma için çeşitli seçenekler vardır: yalnız kelimenin sayımı (count), terim frekansı (term frequency, TF) ve TF-IDF türleri. Hoca TF'i kelimenin sayısının belgede en çok geçen kelimenin sayısına oranı olarak hatırlattı. Slayttaki örnek ağırlık şudur:

$$w_{x,j} = f_{x,j} \cdot \frac{\text{idf}_x}{\max_i \text{idf}_i}$$

$f_{x,j}$, x kelimesinin j belgesindeki frekansıdır. Ters belge frekansı (inverse document frequency) ise şöyledir: N derlemdeki (corpus) toplam belge sayısı, n_x kelimenin geçtiği belge sayısı olmak üzere $\text{idf}_x = \log \frac{N}{n_x}$.

Hocanın sayısal gösterimi: belgede A, B, C kelimeleri olsun. IDF değerleri 5, 3 ve 15 olsun. A'yı ağırlıklandırmak için A'nın frekansı $5/15$ ile çarpılır, çünkü en büyük IDF 15'tir.

14.2 Benzerlik formülleri

İki terimli bir sorgu düşünelim: k_x ve k_y. Örneğin "Yıldız" OR "Davutpaşa". İkisinden birinin geçtiği belgeler gelecek, ama sıralı gelecek. Bunun için her belgeyle benzerlik hesaplanır. w_1 ve w_2, sorgu terimlerinin o belgedeki ağırlıklarıdır.

OR sorgusunda ağırlıkların kareleri toplanır, terim sayısına bölünür, karekök alınır. AND sorgusunda ise birden farkları alınır, kareleri toplanır, terim sayısına bölünür, karekök alınır ve sonuç birden çıkarılır:

$$\text{sim}(q_{or}, d) = \sqrt{\frac{w_1^2 + w_2^2}{2}} \qquad \text{sim}(q_{and}, d) = 1 - \sqrt{\frac{(1-w_1)^2 + (1-w_2)^2}{2}}$$

Hoca "neden 2, neden 3 değil?" sorusunu kendisi sorup genellemeye geçti. Üs p olabilir, $1 \leq p \leq \infty$. Sonsuz alınmaz, ama 2, 3, 4 alınabilir. Terim sayısı t ise:

$$\text{sim}(q_{or}, d) = \left(\frac{w_1^p + \cdots + w_t^p}{t}\right)^{1/p} \qquad \text{sim}(q_{and}, d) = 1 - \left(\frac{(1-w_1)^p + \cdots + (1-w_t)^p}{t}\right)^{1/p}$$

> **Ek bilgi (derste anlatılmadı):** p'nin uç değerleri modeli diğerlerine bağlar. $p = 1$ iken OR ve AND formüllerinin ikisi de ağırlıkların ortalamasını verir. $p \to \infty$ iken OR formülü en büyük ağırlığa, AND formülü en küçük ağırlığa yaklaşır. Bu, sonraki bölümde gelen bulanık mantığın max ve min işlemleridir.

14.3 Karma sorgu

Sorgu $q = (k_1 \text{ AND } k_2) \text{ OR } k_3$ olsun. Her belge için önce AND kısmı hesaplanır:

$$a = 1 - \left(\frac{(1-w_1)^p + (1-w_2)^p}{2}\right)^{1/p}$$

Sonra bu sonuç bir terim gibi OR'a girer:

$$\text{sim}(q, d) = \left(\frac{a^p + w_3^p}{2}\right)^{1/p}$$

D1, D2, D3 için benzerlikler bulunur ve belgeler en benzeyenden en az benzeyene sıralanır.

14.4 Tahtadaki örnek

Hoca, "ağırlıklar nasıl bulunuyor" sorusu takılmasın diye tahtada küçük bir derlem yazdı: D1 = A A A A B, D2 = A B B, D3 = B C D E. Derlemde 3 belge vardır.

- A, D1'de 4 kez geçer. Formülün başındaki frekans budur.
- A 3 belgenin 2'sinde geçer: $3/2 = 1{,}5$.
- B 3 belgenin hepsinde geçer: $3/3 = 1$.
- En büyük değer A'nınkidir, yani formülün paydasına 1,5 yazılır.

15. Vektör Uzayı Modeli

15.1 Terim-belge matrisi

Vektör uzayı modelinin temeli bildiğimiz terim-belge matrisidir. Sütunlar belgeler, satırlar terimlerdir, değerler terimin belgede kaç kez geçtiğidir. Sorgu da aynı terimlerle bir vektör olarak yazılır. Sorgunun D1 ile, D2 ile, ..., DN ile benzerliği hesaplanır ve belgeler büyükten küçüğe sıralanır.

Benzerlik için çoğunlukla kosinüs benzerliği kullanılır, ama başka bir ölçüt de kullanılabilir. Değerler ham sayı yerine ağırlıklandırılmış da olabilir.

> **Hocanın vurgusu:** Ağırlıklandırma formülü serbesttir. Farklı kitaplarda farklı formüller vardır. Bir formüle 1 eklemek senin probleminde daha iyi sonuç veriyorsa ekle. Bu tamamen sana kalmış.

15.2 TF-IDF örneği

100 kelimelik bir belgede "cow" kelimesi 3 kez geçsin. TF, 3'ün toplam terim sayısına oranıdır: $3/100 = 0{,}03$.

10 milyon belge olsun, "cow" bunların 1000'inde geçsin. IDF $= \log(10^7/10^3) = 4$.

TF-IDF $= 0{,}03 \times 4 = 0{,}12$. Bu, "cow" kelimesi için kullanılabilecek ağırlıktır.

15.3 Kosinüs benzerliği

Uzayda iki belge iki vektördür. Benzerlikleri aralarındaki θ açısıyla ölçülür. Hoca açıyı hesaplamakla uğraşmak yerine kosinüs benzerliğinin (cosine similarity) kullanıldığını söyledi. Pay iki vektörün skaler çarpımıdır: karşılıklı elemanlar çarpılıp toplanır. Payda vektör uzunluklarının çarpımıdır:

$$\cos\theta = \frac{A \cdot B}{\|A\|\,\|B\|} = \frac{\sum_i a_i b_i}{\sqrt{\sum_i a_i^2}\,\sqrt{\sum_i b_i^2}}$$

Sonuç 1'e ne kadar yakınsa belgeler o kadar benzerdir.

Örnek 1: $A = (1,1,1)$, $B = (1,0,1)$, $C = (10,5,9)$.

- $A \cdot B = 1+0+1 = 2$, $\|A\| = \sqrt{3}$, $\|B\| = \sqrt{2}$. $\cos(A,B) = \frac{2}{\sqrt{3}\sqrt{2}} \approx 0{,}82$.
- $A \cdot C = 10+5+9 = 24$, $\|C\| = \sqrt{100+25+81} = \sqrt{206}$. $\cos(A,C) = \frac{24}{\sqrt{3}\sqrt{206}} \approx 0{,}97$.

Hoca hesabın sonucunu okumadan bıraktı; sayılar ara adım olarak eklendi. Bkz. Ders Sonrası Notlar.

Örnek 2 (slayt): $D_1 = 2T_1 + 3T_2 + 5T_3$, $D_2 = 3T_1 + 7T_2 + T_3$, $Q = 0T_1 + 0T_2 + 2T_3$.

- $\cos(Q, D_1) = \frac{10}{2\sqrt{38}} \approx 0{,}81$.
- $\cos(Q, D_2) = \frac{2}{2\sqrt{59}} \approx 0{,}13$.

Q, D1'e daha çok benzer. Aradaki açı daha küçüktür.

15.4 İç çarpım

Benzerlik için iç çarpım (inner product) da kullanılabilir: yalnız karşılıklı elemanlar çarpılıp toplanır. Slayttaki örnekte terimler retrieval, database, architecture, computer, text, management ve information'dır. İkili (var/yok) vektörlerde sonuç ortak terim sayısını verir, örnekte 3'tür. Sonuç 0-1 arasına normalize edilebilir.

16. İndeksleme: Ters Çevrilmiş Dosya

Örnek belgeler şunlardır:

- T_0 = "it is what it is"
- T_1 = "what is it"
- T_2 = "it is a banana"

"what", "is", "it" kelimeleriyle arama yapılsın. "what" 0 ve 1'de, "is" 0, 1 ve 2'de, "it" 0, 1 ve 2'de vardır. Kelimeler AND ile bağlandığı için kesişim alınır: $\{0, 1\}$.

Ama bu belgelere nasıl erişilecek? Her aramada belgelerin içine teker teker girilmez. Spider önceden indeks oluşturur.

Ters çevrilmiş dosya indeksi (inverted file index): her kelime için yalnız geçtiği belgelerin adresleri tutulur.

- "a": $\{2\}$, "banana": $\{2\}$
- "is": $\{0, 1, 2\}$, "it": $\{0, 1, 2\}$
- "what": $\{0, 1\}$

Tam ters çevrilmiş dosya indeksi (full inverted file index): belge adresine ek olarak kelimenin belgedeki konumu da tutulur. Kelime aranınca belgedeki yerinin işaretlenebilmesi bundan gelir. Konumlar sıfırdan başlar:

- "a": $\{(2, 2)\}$
- "banana": $\{(2, 3)\}$
- "is": $\{(0, 1), (0, 4), (1, 1), (2, 1)\}$

Veri yapısı olarak her indekslenen kelimeden bir bağlı liste çıktığı düşünülebilir. Listenin her elemanında belge numarası ve o belgedeki konum vardır. Örneğin "computer" 7. belgenin 4. sırasında, 8. belgenin 10. sırasında, başka bir konumda daha geçiyor olabilir. Kaç kez geçiyorsa o kadar adres vardır.

Hoca koskoca webin böyle indekslenmesi için gereken donanıma dikkat çekti, ama webdeki her sayfanın indekslenmiş olmadığını da ekledi.

17. Bulanık Mantık

17.1 Klasik mantık, olasılık ve bulanık mantık

Klasik mantıkta bir şey 0 ya da 1'dir, ya vardır ya yoktur. Olasılıkta değerlerin toplamı her zaman 1'dir. Bulanık (fuzzy) mantıkta ise üyelik vardır: bir şey bir gruba hangi derecede üyedir?

Üç örnek:

- **"Joe uzun":** Joe'nun boyu 1,90 olsun. Hocanın boyu 1,57; ona göre Joe çok uzundur. 1,87 boyundaki biri için ise alt tarafı 3 cm uzundur. Uzunluk kişiden kişiye değişir, göreceli bir kavramdır. Joe'nun "uzun" kümesine üyeliği birine göre 0,8 iken bir basketbolcuya göre 0,2 olabilir.
- **Ekonomi ve siyaset:** Bir yazıda ikisi karışık geçsin. Yazı ekonomiye 0,8, siyasete 0,7 üyelikle dahil edilebilir. Toplam 1,5'tir. Olasılık 1'den büyük olamayacağına göre bu olasılık değildir.
- **"Genç" kümesi:** Ann 28, Bob 35, Charlie 23 yaşında. En küçük olan Charlie kümeyi 1 üyelikle, Ann 0,8 ile, Bob 0,1 ile temsil eder. Toplam 1,9'dur.

> **Hocanın vurgusu:** Olasılık demeyin, üyelik derecesi (degree of membership) deyin. Üyelik derecelerinin toplamı 1'den büyük olabilir. Fuzzy'yi fuzzy yapan budur.

17.2 Yaş grupları grafiği

Hoca, bulanık mantık kitaplarının hepsinde buna benzer bir grafik bulunacağını söyledi:

- **Genç:** 0-25 yaş arası üyelik 1'dir. 25-40 arasında azalır, 40'ta 0 olur.
- **Orta yaş:** 25-55 yaş arasıdır. Ortasında, 40 yaşında üyelik 1'dir. İki yana doğru azalır.
- **Yaşlı:** 40-55 arasında üyelik artar, 55'ten sonra 1'dir.

Grafiğin düşey ekseni üyelik derecesidir. Klasik mantıkta bir kişi ya genç ya orta yaşlı ya yaşlıdır. Burada aynı kişi birden çok kümeye farklı derecede üye olabilir.

17.3 Bulanık işlemler

Konuyla bağlantısı şudur: sorgularda klasik mantık yerine bulanık mantık kullanılırsa belki daha iyi sonuç alınır. Sorgularda AND, OR ve NOT olduğu için bunların karşılığı gerekir. Değerler kelimelerin IDF ya da TF-IDF değerleri olabilir.

$A = (1;\ 0{,}20;\ 0{,}75)$ ve $B = (0;\ 0{,}45;\ 0{,}50)$ olsun:

- **OR (birleşim, maksimum):** $(1;\ 0{,}45;\ 0{,}75)$.
- **AND (kesişim, minimum):** $(0;\ 0{,}20;\ 0{,}50)$.
- **NOT (tümleyen, $1 - x$):** A'nın tümleyeni $(0;\ 0{,}80;\ 0{,}25)$.

18. Karma Min-Maks Modeli

Bulanık işlemleri bilgi erişiminde kullanmak için karma min-maks modeli (mixed min and max model, MMM) geliştirilmiştir. $A_1, \ldots, A_n$ terimler, d_{A_i} ise terimlerin belgedeki ağırlıklarıdır. Model, adından anlaşıldığı gibi hem maksimumu hem minimumu karıştırır:

$$\text{SIM}(Q_{or}, D) = C_{or1} \cdot \max(d_{A_1}, \ldots, d_{A_n}) + C_{or2} \cdot \min(d_{A_1}, \ldots, d_{A_n})$$

$$\text{SIM}(Q_{and}, D) = C_{and1} \cdot \min(d_{A_1}, \ldots, d_{A_n}) + C_{and2} \cdot \max(d_{A_1}, \ldots, d_{A_n})$$

Katsayıların mantığı şudur: OR'da işlem maksimumdur, bu yüzden maksimumun etkisi güçlü olmalıdır, $C_{or1} > C_{or2}$ istenir. AND'de işlem minimumdur, bu yüzden minimumun etkisi güçlü olmalıdır, $C_{and1} > C_{and2}$ istenir.

Hoca bu katsayıların ezbere bilinemeyeceğini, farklı veri setleriyle yapılan çalışmalardan elde edildiğini söyledi:

- $C_{or1} > 0{,}2$, $C_{or2} = 1 - C_{or1}$. Örneğin $C_{or1} = 0{,}8$ ise $C_{or2} = 0{,}2$.
- $C_{and1} \in [0{,}5;\ 0{,}8]$, $C_{and2} = 1 - C_{and1}$.

Bu model standart Boole modeline göre çok daha iyi sonuç verir. Genişletilmiş Boole ile bulanık model, standart Boole'nin dezavantajını ortadan kaldırmak için ortaya atılmıştır: standart modelde sıralama yoktu, diğerlerinde formülden sayısal değer elde edildiği için sıralama yapılabilir.

Dersin Toparlaması

Dersin ilk yarısı metin üzerinde çalışan veri yapıları ve algoritmalara ayrıldı. Trie ortak önekleri paylaşarak hızlı erişim ve bellek verimliliği sağlıyor. Sonek ağacı tek bir dizenin soneklerini saklayarak alt dize aramayı kolaylaştırıyor. LCS ile edit distance tablo tabanlı dinamik programlamayla çözülüyor. İkincisi, yazım hatası düzeltmede işe yarayan sayısal bir uzaklık veriyor.

İkinci yarıda bilgiye erişim anlatıldı. Standart Boole modeli basit ama sıralama yapamıyor. Genişletilmiş Boole, bulanık (MMM) ve vektör uzayı modelleri sonuçları bir sayısal benzerlikle sıraladığı için bu eksiği gideriyor. İndeksleme ise bunların büyük ölçekte çalışmasını sağlıyor.

Sıradaki konu cebirsel modellerden LSA'dır. Hoca bunu önemli bir konu olarak nitelendirdi, ardından değerlendirme ölçütlerine ve muhtemelen bilgi çıkarımına geçileceğini söyledi.

Ders Sonrası Notlar

Ders sonrası not (LCS adı): Hoca LCS'yi "en uzun alt string" diye tanıttı, ama açıklaması alt diziye aittir. Karakterlerin ardışık olması gerekmeyen LCS, alt dizidir (subsequence). Ardışık olan "en uzun ortak alt dize" (longest common substring) ayrı bir problemdir.

Ders sonrası not (Y dizesi): Altyazıda Y dizesi "A, T, C, G, G, G" diye geçiyor. Bu dizeyle LCS uzunluğu 4 çıkıyor ("ACGG"). Hocanın bulduğu 5 ve "ATTGG" yalnız Y = ATTCGG ile tutuyor. Python ile iki dize de denendi; doğrusu ATTCGG.

Ders sonrası not (karmaşıklık): Altyazıda sonek ağacı aramasının karmaşıklığı iki yerde de "O(m)" geçiyor. n uzunluğundaki kalıbı aramak $O(n)$ sürer. Edit distance için söylenen "n kare" ise m ve n boylu dizeler için $O(mn)$'dir.

Ders sonrası not (Levenshtein): Hoca Levenshtein metriğinde yerine koymanın 2 maliyetli olduğunu ve genelde bunun kullanıldığını söyledi. Bu, kullanılan biçimlerden biridir. Levenshtein uzaklığının yaygın tanımında ekleme, silme ve yerine koymanın üçü de 1 maliyetlidir. Bu tanımla "intention"-"execution" uzaklığı 5'tir. Hangi maliyetin kullanıldığı belirtilmelidir. Python ile iki durum da hesaplandı: 5 ve 8.

Ders sonrası not (satır ve kolon): Hoca bir yerde "satırdaki işlemler silme, kolondaki işlemler ekleme" dedi, geri izlemede ise "satırda gitmek ekleme, kolonda inmek silme" dedi. Kaynak dize düşey eksende, hedef dize yatay eksende olduğu için doğrusu ikincisidir: yatay ilerlemek hedefe harf ekler, düşey ilerlemek kaynaktan harf siler.

Ders sonrası not (kosinüs örneği): Hoca "A vektörü B'ye mi, C'ye mi benzer?" sorusuna "tabii ki B'ye benzer" dedi ve hesabın sonucunu okumadı. Hesap tamamlanınca $\cos(A, C) \approx 0{,}97$, $\cos(A, B) \approx 0{,}82$ çıkıyor. A, C'ye daha çok benzer. Kosinüs benzerliği vektörlerin büyüklüğüne değil yönüne bakar.

Ders sonrası not (açı): Hoca "açıyı hesaplamak zor, onun yerine kosinüs benzerliği kullanılıyor" dedi. Kosinüs benzerliği zaten bu açının kosinüsüdür. Kolaylık, açının kendisini bulmaya gerek kalmamasıdır.

Ders sonrası not (genişletilmiş Boole ağırlıkları): Tahtadaki örnekte IDF logaritma alınmadan N/n olarak hesaplandı (A için 1,5, B için 1). Formülde logaritma vardır; logaritmayla B'nin IDF'si 0 olur. Ayrıca frekans ham sayı olarak (4) alındı, bu durumda A'nın ağırlığı $4 \times 1{,}5/1{,}5 = 4$ çıkar. AND formülündeki $1 - w$ ifadesi ağırlıkların 0-1 arasında olmasını varsayar. Bu yüzden formüldeki $f_{x,j}$ normalize edilmiş frekans olarak düşünülmelidir.

Ders sonrası not (karma sorgu): Hoca $(k_1$ AND $k_2)$ OR k_3 sorgusunda "K1 ve K2 kesin olacak, K3 olabilir de olmayabilir de" dedi. Mantıksal olarak OR, yalnız k_3 içeren belgeleri de kabul eder. Hesaplanan formül de bu belgelere sıfır olmayan benzerlik verir.

Ders sonrası not (TF tanımı): Hoca TF'i önce en sık geçen kelimeye oranla tanımladı, "cow" örneğinde ise toplam terim sayısına böldü. İkisi de kullanılan normalizasyonlardır, ama aynı şey değildir.

Ders sonrası not (bulanık kesişim): Hoca kesişimi önce "1; 0,20; 0,50" diye okudu, sonra sonucun çıkmadığını fark edip düzeltti. İlk eleman $\min(1, 0) = 0$'dır: $(0;\ 0{,}20;\ 0{,}50)$. Ayrıca orta yaş kümesi için "1 ile 55 arasında bir değer" dedi; kastı 1 ile 0 arasında bir üyelik derecesidir.

Ders sonrası not (MMM katsayıları): Hoca hem $C_{or1} > C_{or2}$ hem $C_{or1} > 0{,}2$ dedi. İkisi tam örtüşmez: $C_{or1} = 0{,}3$ ikinci koşula uyar, ama $C_{or2} = 0{,}7$ olur. Ders kapsamında asıl mantık, OR'da maksimumun etkisinin büyük olmasıdır.

Ders sonrası not (sıradaki konu): Hoca sıradaki konuyu bir yerde “LSTM” diye söyledi. Ders sonunda bunun LSA (gizli anlamsal analiz, latent semantic analysis) olduğu anlaşılıyor.

Ders sonrası not (yazımlar): Altyazıda bozuk geçen terimlerin doğruları: “Bullun/boğulun” Boolean, “kosan simularite” cosine similarity, “Lievenstein” Levenshtein, “safiks tri” suffix tree, “fazi” fuzzy, “TRI/tria/treya” trie, “kubik” QBIC, “Webseq” WebSEEk, “AskGives” Ask Jeeves, “AnswerBuzz” AnswerBus, “Hidenmarko” Hidden Markov. Altyazıda “NLP 28 yaşıyla” geçen yer Ann’dir. “It is what is it” diye geçen belge, konum bilgisine göre “it is what it is”tir.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.